ProofPass

A Zero-Knowledge Compliance Layer for Real-World Assets on Stellar

Stellar Hacks: Real-World ZK | July 2026

Abstract

Financial compliance today runs on document exchange. To prove eligibility for regulated financial products, individuals hand over bank statements, tax returns, and identity documents to institutions that store them indefinitely — creating privacy risk for users and liability risk for institutions.

ProofPass replaces document exchange with cryptographic proof. Users generate a zero-knowledge proof that a signed, authoritative document satisfies a regulatory requirement — without revealing the document itself. Institutions verify the proof on-chain and grant access automatically. No documents change hands. No sensitive data is stored.

Our first implementation targets SEC accredited investor verification for tokenized Real-World Asset (RWA) yield pools on Stellar.

---

1. The Problem

1.1 The Human Problem

Every day, financial institutions ask people to hand over their most sensitive documents — not because they want them, but because regulations require proof of eligibility. The result is a compliance process built on trust, paperwork, and centralized databases full of sensitive information that becomes an attractive target for breach.

1.2 The Regulatory Reality

Under SEC Regulation D, Rule 501(a), only accredited investors — those with a net worth exceeding $1,000,000, excluding the value of their primary residence — may access certain private investment products, including tokenized RWAs. This is a hard legal gate, not an optional check.

1.3 The Infrastructure Gap

Stellar is becoming a settlement layer for institutional tokenized assets. Franklin Templeton, WisdomTree, and DTCC are live or incoming on the network, and Stellar's RWA market has grown nearly 200% to over $580 million in tokenized assets. The settlement infrastructure exists. The compliant, privacy-preserving investor verification layer does not.

1.4 The Alignment

The investor must prove their wealth to access yield. The protocol must verify wealth to avoid regulatory liability. Neither party wants the underlying documents to actually change hands. This is precisely the problem zero-knowledge proofs are designed to solve.

---

2. Why This Isn't Another Private Compliance Demo

Most zero-knowledge compliance projects prove that a user typed "true" into a form. The circuit verifies a self-reported claim — inputs like `kycPassed` or `sanctionsClear` supplied directly by the user, with no external authority backing them. It is a checkbox wrapped in cryptography.

ProofPass proves something categorically different: that a real, cryptographically signed document — issued by a licensed professional and chained back to a recognized public certificate authority — actually contains the statutory language the user claims it does. We do not ask "do you say you're compliant." We ask "can you prove a trusted third party already verified this," and check the cryptographic math ourselves.

This distinction is the core of ProofPass's design and its primary differentiation from comparable ZK compliance projects in this ecosystem.

---

3. The Attestation Source

3.1 Why CPA Attestation Letters

To prove accredited investor status in the real world, investors commonly obtain a standardized letter signed by a licensed CPA, attorney, or registered broker-dealer. When issued, this letter is signed electronically via DocuSign, embedding a PKCS#7 digital signature into the PDF to comply with the federal ESIGN Act.

This approach was chosen after evaluating and ruling out three alternatives:

- **zkTLS** (session interception from brokerage websites): introduces third-party notary trust assumptions, faces active anti-automation countermeasures from brokerages, and requires complex browser-level tooling not achievable within the build timeline.
- **ZK Email** (brokerage balance notification emails): brokerage notification emails typically do not contain the actual account balance in the email body — the data lives behind a login wall in a PDF.
- **Raw brokerage PDF statements**: not cryptographically signed, and therefore forgeable without a trust anchor.

CPA attestation letters solve all three problems simultaneously: the signature is guaranteed and standardized (DocuSign/PKCS#7), the language is legally mandated (SEC Rule 501(a)), and this is already how accredited investor verification works in TradFi today — ProofPass makes the existing process trustless rather than inventing a new one.

3.2 The Statutory Anchor Phrase

SEC Rule 501(a) mandates a specific exclusion: home equity cannot count toward the $1M net worth threshold. Every legitimate CPA attestation letter must therefore contain the phrase:

> *"...excluding the value of their primary residence."*

Any deviation exposes the issuing CPA to liability. This makes the phrase a reliable, legally-mandated anchor for our proof circuit to target — CPAs do not get creative with this language.

3.3 Trust Model

The trust anchor is **DigiCert Trusted Root G4**, the certificate authority chain DocuSign's eSignature platform uses to sign documents. This is a self-signed, publicly recognized root certificate — the same class of trust anchor a web browser uses to validate HTTPS. No notary network, no relay server, no third-party trust assumption beyond a certificate authority already trusted throughout the internet.

---

4. System Architecture

4.1 End-to-End Flow

```
Investor obtains DocuSign-signed CPA Attestation PDF
        ↓
Investor runs local prover (SP1 zkVM) against the PDF
        ↓
Inside the zkVM, the program:
  1. Verifies the PKCS#7 signature via a 7-stage crypto pipeline
     (ByteRange extraction → CMS/SignedData decoding →
      RSA-PKCS1v15-SHA256 verification → full certificate
      chain walk to the hardcoded DigiCert Trusted Root G4)
  2. Confirms the document is dated within 90 days (SEC recency requirement)
  3. Parses the PDF text layer and matches the SEC anchor phrase
  4. Derives a nullifier from the signature bytes (prevents replay)
        ↓
STARK proof generated locally
        ↓
SP1 compresses the STARK into a Groth16 proof over BN254
(critical: raw STARK proofs exceed Soroban's 100M WASM instruction limit)
        ↓
Proof + public inputs (investor wallet, nullifier, timestamp)
encoded for Soroban and written to proof.json
        ↓
Investor connects wallet, submits proof to the Soroban contract
        ↓
Contract verifies the Groth16 BN254 proof using Stellar Protocol 26
native BN254 host functions (pairing checks run at host level,
not consuming the contract's WASM instruction budget)
        ↓
Contract checks the nullifier has not been used, stores it,
stores a 5-year credential, and calls set_authorized on the
Stellar Asset Contract
        ↓
Investor's AUTH_REQUIRED trustline flips to true
        ↓
Investor can now access the tokenized RWA yield pool
```

4.2 Why SP1 + Groth16 BN254

SP1 is a zero-knowledge virtual machine that allows standard, readable Rust code to be compiled into a zero-knowledge proof — avoiding the need to hand-write PDF parsing and cryptographic verification logic in a low-level circuit language.

Raw SP1 output is a STARK proof: fast to generate, but too large to verify inside a Soroban smart contract without exceeding the WASM CPU limit. SP1's native Groth16 wrapper compresses this into a compact proof (~260 bytes) over the BN254 elliptic curve, which Stellar's Protocol 25 ("X-Ray") and Protocol 26 ("Yardstick") upgrades

added native host function support for — meaning the heaviest cryptographic operations (pairing checks) run at the network level rather than inside the contract's constrained execution budget.

Noir + UltraHonk was considered and rejected: UltraHonk proofs are larger and more expensive to verify on-chain, which matters significantly given that PDF parsing inside a zkVM is already computationally heavy.

4.3 Soroban Contract Storage Architecture

Soroban state bloat can exhaust a contract's execution budget. ProofPass is deliberately minimalist:

Data	Storage Type	Rationale
Groth16 verification key	Instance	Global configuration, loaded on every contract invocation
Nullifier hashes	Persistent (TTL-extended)	Must persist indefinitely to prevent the same attestation letter from being submitted twice
Investor credentials + expiry	Persistent (TTL-extended)	5-year validity period must outlive default ledger TTL
Investor authorization state	Not contract storage	Delegated entirely to the Stellar Asset Contract via `set_authorized`

ProofPass does not maintain an internal list of approved investors. Authorization state lives natively on the Stellar network via the Asset Contract's trustline mechanism.

4.4 Compliance Design

SEC Requirement	Implementation
Net worth > $1,000,000, excluding primary residence	Statutory anchor phrase match inside the zkVM
Documents must be dated within approximately 90 days	Circuit verifies PDF creation date against current timestamp
Verification remains valid for approximately 5	Credential expiry encoded directly

years before re-verification	into contract storage

---

5. Competitive Landscape

Comparable zero-knowledge compliance projects in the current hackathon ecosystem verify self-reported inputs against a sample or placeholder circuit — meaning the "proof" confirms only that a user asserted compliance, not that any real-world authority verified it. ProofPass's differentiation is structural, not cosmetic: the proof is only as meaningful as the document behind it, and ProofPass is the only implementation in this space that cryptographically verifies a real signed document against a real, publicly trusted certificate authority.

---

6. Vision

(Roadmap language — beyond current build scope.)

The accredited investor gate is ProofPass's first implementation, not its ceiling. The same primitive — proving a signed document satisfies a policy without revealing it — generalizes to:

- AML and KYC verification
- Proof of residency or jurisdiction
- DAO and organizational membership
- Professional licensing verification
- Institutional compliance and regulatory reporting

Anywhere a system asks "can you prove this," ProofPass allows the answer "yes" — without requiring everything else to be revealed in the process.

---

7. Closing

The internet was built around sharing documents. Regulated finance should be built around sharing proofs.

ProofPass replaces document exchange with cryptographic proof — letting compliant users access regulated financial products without sacrificing their privacy.

Appendix: Technical Reference

- **SP1 zkVM:** `github.com/succinctlabs/sp1`
- **Soroban Groth16 verifier reference:** `github.com/stellar/soroban-examples/tree/main/groth16_verifier`
- **BN254 proof format reference (zkVM-compatible):** `github.com/NethermindEth/stellar-risc0-verifier`
- **Trust anchor: DigiCert Trusted Root G4 (self-signed, expires January 2038)**
- **Attestation signer: DocuSign eSignature platform (PKCS#7 / ESIGN Act compliant)**